

Week 9: Graded Mini Project

Project title: House Sales Data Analysis

Executive summary: Analyse housing data to identify key factors influencing sale prices through exploratory analysis, feature engineering, and predictive modeling. This project focus on predicting employee attrition using split, train and testing the given data set. The goal is to generate a consolidated report on attrition driving factors, so that HR can take action. 3 models are used. The best performance model is selected in the end.

Dataset Overview:

Dataset: House sales data

Records: 1460

Target Variable: Sales Price

Features: - LivArea_Qual, TotalArea, House_Age, Garage_Efficiency, Quality_Index

Data Preprocessing: Data cleaning and preprocessing done. Feature engineering done. EDA analysis, visual reports done.

Final Report:

Model is built and the best model is XGBoost.

Following are the details of the code and output of various data preprocessing tasks, model building, model evaluation, saving the best model to pickle file, report generation and analytics derived based on the modeling. The analytics will help to determine key factors that are influencing sales price and enable the user in decision making.

Task 1: Data Cleaning and Preprocessing

1.1 Handling Missing Values: Identify and handle missing values appropriately.

```
def handle_missing_values(self):  
  
  
    df = self.df.copy()  
  
  
    print("\n==== MISSING VALUE REPORT (BEFORE) =====\n")  
  
    missing_before = df.isnull().sum()  
    missing_before = missing_before[missing_before > 0]  
  
    if len(missing_before) == 0:  
        print("No missing values found.")  
  
    return df
```

```
missing_percent = (missing_before / len(df)) * 100
```

```
report = pd.DataFrame({
```

```
    "Missing Count": missing_before,
```

```
    "Missing %": missing_percent
```

```
}).sort_values(by="Missing %", ascending=False)
```

```
print(report)
```

```
print("\n===== HANDLING MISSING VALUES =====\n")
```

```
num_cols = df.select_dtypes(include=['number']).columns
```

```
cat_cols = df.select_dtypes(include=['object']).columns
```

```
# ---- Categorical handling ----
```

```
if len(cat_cols) > 0:
```

```
    print(f"Categorical columns filled with 'None': {list(cat_cols)}")
```

```
    df[cat_cols] = df[cat_cols].fillna("None")
```

```
# ---- Numerical handling ----
```

```
if len(num_cols) > 0:
```

```
    print(f"Numerical columns filled with 0: {list(num_cols)}")
```

```
    df[num_cols] = df[num_cols].fillna(0)
```

```
self.df = df
```

```
print("\n===== MISSING VALUE REPORT (AFTER) =====\n")
```

```
missing_after = df.isnull().sum()
```

```
missing_after = missing_after[missing_after > 0]
```

```
if len(missing_after) == 0:
```

```
    print("All missing values handled successfully.")
```

```
else:
```

```
    print(missing_after)
```

```
return df
```

The output is as follows:

===== MISSING VALUE REPORT (BEFORE) =====

	Missing Count	Missing %
PoolQC	1453	99.520548
MiscFeature	1406	96.301370
Alley	1369	93.767123
Fence	1179	80.753425
MasVnrType	872	59.726027
FireplaceQu	690	47.260274
LotFrontage	259	17.739726
GarageType	81	5.547945
GarageYrBlt	81	5.547945
GarageFinish	81	5.547945
GarageQual	81	5.547945
GarageCond	81	5.547945
BsmtExposure	38	2.602740
BsmtFinType2	38	2.602740
BsmtQual	37	2.534247
BsmtCond	37	2.534247
BsmtFinType1	37	2.534247
MasVnrArea	8	0.547945

Electrical 1 0.068493

===== HANDLING MISSING VALUES =====

Categorical columns filled with 'None': ['MSZoning', 'Street', 'Alley', 'LotShape', 'LandContour', 'Utilities', 'LotConfig', 'LandSlope', 'Neighborhood', 'Condition1', 'Condition2', 'BldgType', 'HouseStyle', 'RoofStyle', 'RoofMatl', 'Exterior1st', 'Exterior2nd', 'MasVnrType', 'ExterQual', 'ExterCond', 'Foundation', 'BsmtQual', 'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFinType2', 'Heating', 'HeatingQC', 'CentralAir', 'Electrical', 'KitchenQual', 'Functional', 'FireplaceQu', 'GarageType', 'GarageFinish', 'GarageQual', 'GarageCond', 'PavedDrive', 'PoolQC', 'Fence', 'MiscFeature', 'SaleType', 'SaleCondition']

Numerical columns filled with 0: ['Id', 'MSSubClass', 'LotFrontage', 'LotArea', 'OverallQual', 'OverallCond', 'YearBuilt', 'YearRemodAdd', 'MasVnrArea', 'BsmtFinSF1', 'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF', '1stFlrSF', '2ndFlrSF', 'LowQualFinSF', 'GrLivArea', 'BsmtFullBath', 'BsmtHalfBath', 'FullBath', 'HalfBath', 'BedroomAbvGr', 'KitchenAbvGr', 'TotRmsAbvGrd', 'Fireplaces', 'GarageYrBlt', 'GarageCars', 'GarageArea', 'WoodDeckSF', 'OpenPorchSF', 'EnclosedPorch', '3SsnPorch', 'ScreenPorch', 'PoolArea', 'MiscVal', 'MoSold', 'YrSold', 'SalePrice']

===== MISSING VALUE REPORT (AFTER) =====

All missing values handled successfully.

1.2 Converting Categorical Variables: Encode categorical variables using suitable methods (e.g., one-hot encoding).

```
# =====

# 3. Encoding Categorical Variables

# =====

def encode_categorical(self):

    df = self.df.copy()

    cat_cols = df.select_dtypes(include=['object']).columns

    print("\n===== CATEGORICAL ENCODING REPORT (BEFORE) =====\n")
    print(f"Number of categorical columns: {len(cat_cols)}")
    print(f"Categorical columns: {list(cat_cols)}")
```

```
if len(cat_cols) == 0:

    print("No categorical columns found.")

    return df


# Show sample before encoding

print("\nSample data (before encoding):\n")

print(df[cat_cols].head())


# One-hot encoding

df_encoded = pd.get_dummies(df, columns=cat_cols, drop_first=True)


print("\n===== AFTER ENCODING =====\n")

print(f"New dataset shape: {df_encoded.shape}")


new_cols = set(df_encoded.columns) - set(df.columns)


print(f"Number of new encoded columns: {len(new_cols)}")


print("\nSample new columns created:")

print(list(new_cols)[:10]) # show first 10 only


self.df = df_encoded


print("\nCategorical encoding completed successfully.")


return df_encoded
```

The output is as follows:

===== CATEGORICAL ENCODING REPORT (BEFORE) =====

Number of categorical columns: 43

Categorical columns: ['MSZoning', 'Street', 'Alley', 'LotShape', 'LandContour', 'Utilities', 'LotConfig', 'LandSlope', 'Neighborhood', 'Condition1', 'Condition2', 'BldgType', 'HouseStyle', 'RoofStyle', 'RoofMatl', 'Exterior1st', 'Exterior2nd', 'MasVnrType', 'ExterQual', 'ExterCond', 'Foundation', 'BsmtQual', 'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFinType2', 'Heating', 'HeatingQC', 'CentralAir', 'Electrical', 'KitchenQual', 'Functional', 'FireplaceQu', 'GarageType', 'GarageFinish', 'GarageQual', 'GarageCond', 'PavedDrive', 'PoolQC', 'Fence', 'MiscFeature', 'SaleType', 'SaleCondition']

Sample data (before encoding):

MSZoning Street Alley LotShape LandContour Utilities LotConfig LandSlope \

0	RL	Pave	None	Reg	Lvl	AllPub	Inside	Gtl
1	RL	Pave	None	Reg	Lvl	AllPub	FR2	Gtl
2	RL	Pave	None	IR1	Lvl	AllPub	Inside	Gtl
3	RL	Pave	None	IR1	Lvl	AllPub	Corner	Gtl
4	RL	Pave	None	IR1	Lvl	AllPub	FR2	Gtl

Neighborhood Condition1 ... GarageType GarageFinish GarageQual GarageCond \

0	CollgCr	Norm	...	Attchd	RFn	TA	TA
1	Veenker	Feedr	...	Attchd	RFn	TA	TA
2	CollgCr	Norm	...	Attchd	RFn	TA	TA
3	Crawfor	Norm	...	Detchd	Unf	TA	TA
4	NoRidge	Norm	...	Attchd	RFn	TA	TA

PavedDrive PoolQC Fence MiscFeature SaleType SaleCondition

0	Y	None	None	None	WD	Normal
1	Y	None	None	None	WD	Normal
2	Y	None	None	None	WD	Normal
3	Y	None	None	None	WD	Abnorml
4	Y	None	None	None	WD	Normal

[5 rows x 43 columns]

===== AFTER ENCODING =====

New dataset shape: (1460, 262)

Number of new encoded columns: 224

Sample new columns created:

['GarageType_Detachd', 'Neighborhood_Timber', 'MiscFeature_None', 'GarageType_Basment', 'BsmtExposure_No', 'Neighborhood_Somerst', 'Exterior2nd_MetalSd', 'GarageType_Attchd', 'Functional_Mod', 'RoofStyle_Mansard']

Categorical encoding completed successfully.

1.3 Normalising Numerical Features: Normalise numerical features to ensure they are on a comparable scale.

```
# =====
# 4. Normalization
# =====

def normalize_data(self):

    df = self.df.copy()

    num_cols = df.select_dtypes(include=['number']).columns

    print("\n===== NORMALIZATION REPORT (BEFORE) =====\n")

    print(f"Number of numerical columns: {len(num_cols)}")
    print(f"Columns: {list(num_cols)}")

    # Show summary statistics before scaling

    print("\nStatistical summary (before normalization):\n")
```

```
print(df[num_cols].describe().T[['min', 'max', 'mean']].head(10))
```

```
if len(num_cols) == 0:
```

```
    print("No numerical columns found.")
```

```
    return df
```

```
scaler = MinMaxScaler()
```

```
df_scaled = df.copy()
```

```
df_scaled[num_cols] = scaler.fit_transform(df[num_cols])
```

```
print("\n===== AFTER NORMALIZATION =====\n")
```

```
print("\nStatistical summary (after normalization):\n")
```

```
print(df_scaled[num_cols].describe().T[['min', 'max', 'mean']].head(10))
```

```
# Verify scaling correctness
```

```
print("\nRange check (should be 0 to 1):")
```

```
print(df_scaled[num_cols].agg(['min', 'max']).head(5))
```

```
self.df = df_scaled
```

```
print("\nNormalization completed successfully.")
```

```
return df_scaled
```

The Output is as follows:

```
===== NORMALIZATION REPORT (BEFORE) =====
```


Number of numerical columns: 38

Columns: ['Id', 'MSSubClass', 'LotFrontage', 'LotArea', 'OverallQual', 'OverallCond', 'YearBuilt', 'YearRemodAdd', 'MasVnrArea', 'BsmtFinSF1', 'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF', '1stFlrSF', '2ndFlrSF', 'LowQualFinSF', 'GrLivArea', 'BsmtFullBath', 'BsmtHalfBath', 'FullBath', 'HalfBath', 'BedroomAbvGr', 'KitchenAbvGr', 'TotRmsAbvGrd', 'Fireplaces', 'GarageYrBlt', 'GarageCars', 'GarageArea', 'WoodDeckSF', 'OpenPorchSF', 'EnclosedPorch', '3SsnPorch', 'ScreenPorch', 'PoolArea', 'MiscVal', 'MoSold', 'YrSold', 'SalePrice']

Statistical summary (before normalization):

	min	max	mean
Id	1.0	1456.0	744.190909
MSSubClass	20.0	120.0	47.420455
LotFrontage	30.0	124.0	69.813636
LotArea	3010.0	16285.0	9224.804545
OverallQual	2.0	10.0	6.379545
OverallCond	4.0	7.0	5.359091
YearBuilt	1910.0	2009.0	1985.020455
YearRemodAdd	1950.0	2010.0	1990.397727
MasVnrArea	0.0	423.0	79.529545
BsmtFinSF1	0.0	1619.0	423.672727

===== AFTER NORMALIZATION =====

Statistical summary (after normalization):

	min	max	mean
Id	0.0	1.0	0.510784
MSSubClass	0.0	1.0	0.274205
LotFrontage	0.0	1.0	0.423549
LotArea	0.0	1.0	0.468159
OverallQual	0.0	1.0	0.547443
OverallCond	0.0	1.0	0.453030

YearBuilt 0.0 1.0 0.757782

YearRemodAdd 0.0 1.0 0.673295

MasVnrArea 0.0 1.0 0.188013

BsmtFinSF1 0.0 1.0 0.261688

Range check (should be 0 to 1):

	Id	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	\
min	0.0	0.0	0.0	0.0	0.0	0.0	
max	1.0	1.0	1.0	1.0	1.0	1.0	

	YearBuilt	YearRemodAdd	MasVnrArea	BsmtFinSF1	...	WoodDeckSF	\
min	0.0	0.0	0.0	0.0	...	0.0	
max	1.0	1.0	1.0	1.0	...	1.0	

	OpenPorchSF	EnclosedPorch	3SsnPorch	ScreenPorch	PoolArea	MiscVal	\
min	0.0	0.0	0.0	0.0	0.0	0.0	
max	1.0	0.0	0.0	0.0	0.0	0.0	

	MoSold	YrSold	SalePrice
min	0.0	0.0	0.0
max	1.0	1.0	1.0

[2 rows x 38 columns]

Normalization completed successfully.

1.4 Outlier Detection: Identify and handle outliers in the dataset.

```
# =====
# 5. Outlier Detection (IQR Method)
# =====
def remove_outliers(self):
```

```
df = self.df.copy()
```

```
print("\n===== OUTLIER DETECTION REPORT (BEFORE) =====\n")
```

```
print(f"Initial dataset shape: {df.shape}")
```

```
num_cols = df.select_dtypes(include=['number']).columns
```

```
outlier_summary = {}
```

```
for col in num_cols:
```

```
    Q1 = df[col].quantile(0.25)
```

```
    Q3 = df[col].quantile(0.75)
```

```
    IQR = Q3 - Q1
```

```
    lower_bound = Q1 - 1.5 * IQR
```

```
    upper_bound = Q3 + 1.5 * IQR
```

```
    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
```

```
    outlier_summary[col] = len(outliers)
```

```
    # Remove outliers for this column
```

```
    df = df[(df[col] >= lower_bound) & (df[col] <= upper_bound)]
```

```
print("\nOutlier count per column:\n")
```

```
for col, count in outlier_summary.items():
```

```
    if count > 0:
```

```
        print(f"{col}: {count} outliers removed")
```

```
print("\n==== AFTER OUTLIER REMOVAL ==== \n")
```

```
print(f"Final dataset shape: {df.shape}")
```

```
removed_rows = self.df.shape[0] - df.shape[0]
```

```
print(f"Total rows removed: {removed_rows}")
```

```
self.df = df
```

```
print("\nOutlier removal completed using IQR method.")
```

```
return df
```

The output is as follows:

```
===== OUTLIER DETECTION REPORT (BEFORE) =====
```

Initial dataset shape: (1460, 262)

Outlier count per column:

MSSubClass: 103 outliers removed

LotFrontage: 272 outliers removed

LotArea: 43 outliers removed

OverallQual: 1 outliers removed

OverallCond: 84 outliers removed

YearBuilt: 4 outliers removed

MasVnrArea: 53 outliers removed

BsmtFinSF1: 1 outliers removed

BsmtFinSF2: 94 outliers removed

```
BsmtUnfSF: 13 outliers removed
TotalBsmtSF: 26 outliers removed
1stFlrSF: 2 outliers removed
LowQualFinSF: 11 outliers removed
GrLivArea: 17 outliers removed
BsmtFullBath: 1 outliers removed
BsmtHalfBath: 28 outliers removed
BedroomAbvGr: 6 outliers removed
KitchenAbvGr: 17 outliers removed
TotRmsAbvGrd: 2 outliers removed
Fireplaces: 2 outliers removed
GarageYrBlt: 31 outliers removed
GarageCars: 1 outliers removed
GarageArea: 9 outliers removed
WoodDeckSF: 6 outliers removed
OpenPorchSF: 25 outliers removed
EnclosedPorch: 94 outliers removed
3SsnPorch: 9 outliers removed
ScreenPorch: 45 outliers removed
PoolArea: 1 outliers removed
MiscVal: 10 outliers removed
SalePrice: 9 outliers removed

===== AFTER OUTLIER REMOVAL =====

Final dataset shape: (440, 262)
Total rows removed: 1020

Outlier removal completed using IQR method.
```

Task 2: Feature Engineering

2.1 Creating Interaction Features: Create meaningful interaction features that may help improve model performance.

```
# =====  
  
# 2.1 Interaction Features  
  
# =====  
  
def create_interaction_features(self):  
  
  
    df = self.df.copy()  
  
  
    print("\n==== INTERACTION FEATURE ENGINEERING (BEFORE) =====\n")  
    print(f"Initial dataset shape: {df.shape}")  
    print(f"Initial columns count: {len(df.columns)}")  
  
  
    created_features = []  
  
  
    # -----  
    # GrLivArea × OverallQual  
    # -----  
    if 'GrLivArea' in df.columns and 'OverallQual' in df.columns:  
        df['LivArea_Qual'] = df['GrLivArea'] * df['OverallQual']  
        created_features.append('LivArea_Qual')  
  
  
    # -----  
    # Total House Area  
    # -----  
    if 'TotalBsmntSF' in df.columns and '1stFlrSF' in df.columns:  
        df['TotalArea'] = df['TotalBsmntSF'] + df['1stFlrSF']  
        created_features.append('TotalArea')  
  
  
    # -----  
    # House Age (remodel-based)
```

```
# -----  
  
if 'YearBuilt' in df.columns and 'YearRemodAdd' in df.columns:  
  
    df['House_Age'] = df['YearRemodAdd'] - df['YearBuilt']  
  
    created_features.append('House_Age')
```

```
# -----
```

```
# Garage Efficiency
```

```
# -----
```

```
if 'GarageArea' in df.columns and 'GarageCars' in df.columns:  
  
    df['Garage_Efficiency'] = df['GarageArea'] * df['GarageCars']  
  
    created_features.append('Garage_Efficiency')
```

```
# -----
```

```
# Quality Index
```

```
# -----
```

```
if 'OverallQual' in df.columns and 'OverallCond' in df.columns:  
  
    df['Quality_Index'] = df['OverallQual'] * df['OverallCond']  
  
    created_features.append('Quality_Index')
```

```
print("\n==== INTERACTION FEATURE REPORT =====\n")
```

```
if len(created_features) == 0:  
  
    print("No interaction features were created.")  
  
else:  
  
    print(f"Number of new features created: {len(created_features)}")  
  
    print("Created features:")  
  
    for feat in created_features:  
  
        print(f"- {feat}")
```

```
print("\n===== AFTER FEATURE ENGINEERING =====\n")
```

```
print(f"Final dataset shape: {df.shape}")
```

```
print(f"New total columns: {len(df.columns)}")
```

```
self.df = df
```

```
print("\nInteraction feature engineering completed successfully.")
```

```
return df
```

The output is as follows:

```
===== INTERACTION FEATURE ENGINEERING (BEFORE) =====
```

Initial dataset shape: (440, 262)

Initial columns count: 262

```
===== INTERACTION FEATURE REPORT =====
```

Number of new features created: 5

Created features:

- LivArea_Qual
- TotalArea
- House_Age
- Garage_Efficiency
- Quality_Index

```
===== AFTER FEATURE ENGINEERING =====
```

Final dataset shape: (440, 267)

New total columns: 267

Interaction feature engineering completed successfully.

2.2 Feature Binning: Bin continuous variables into discrete categories for better interpretability.

```
# =====  
  
# 2.2 Feature Binning  
  
# =====  
  
def bin_features(self):  
  
    df = self.df.copy()  
  
    print("\n==== FEATURE BINNING (BEFORE) ==== \n")  
    print(f"Initial dataset shape: {df.shape}")  
    print(f"Initial columns count: {len(df.columns)}")  
  
    created_bins = []  
  
    # -----  
    # Bin House Age  
    # -----  
    if 'YearBuilt' in df.columns:  
  
        df['HouseAge'] = 2026 - df['YearBuilt']  
  
        df['Age_Bin'] = pd.cut(  
            df['HouseAge'],  
            bins=[-1, 10, 25, 50, 100, 200],
```

```
labels=['New', 'Recent', 'MidAge', 'Old', 'VeryOld']
```

```
)
```

```
created_bins.append('Age_Bin')
```

```
# -----
```

```
# Bin Living Area
```

```
# -----
```

```
if 'GrLivArea' in df.columns:
```

```
df['LivArea_Bin'] = pd.qcut(
```

```
df['GrLivArea'],
```

```
q=4,
```

```
labels=['Low', 'Medium', 'High', 'VeryHigh']
```

```
)
```

```
created_bins.append('LivArea_Bin')
```

```
# -----
```

```
# Bin Lot Area
```

```
# -----
```

```
if 'LotArea' in df.columns:
```

```
df['LotArea_Bin'] = pd.qcut(
```

```
df['LotArea'],
```

```
q=4,
```

```
labels=['Small', 'Medium', 'Large', 'VeryLarge']
```

```
)
```

```
created_bins.append('LotArea_Bin')
```

```
print("\n==== BINNING SUMMARY ==== \n")
```

```
if len(created_bins) == 0:
```

```
    print("No binning features were created.")
```

```
else:
```

```
    print(f"Number of new binned features: {len(created_bins)}")
```

```
    for feat in created_bins:
```

```
        print(f"- {feat}")
```

```
# -----
```

```
# One-hot encoding of bins
```

```
# -----
```

```
df_before_encoding = df.copy()
```

```
df = pd.get_dummies(df, drop_first=True)
```

```
print("\n==== AFTER FEATURE BINNING + ENCODING ==== \n")
```

```
print(f"Final dataset shape: {df.shape}")
```

```
print(f"New columns added: {df.shape[1] - self.df.shape[1]}")
```

```
print("\nSample of newly created columns:")
```

```
new_cols = set(df.columns) - set(self.df.columns)
```

```
print(list(new_cols)[:10])
```

```
self.df = df
```

```
print("\nFeature binning completed successfully.")
```

```
return df
```

The output is as follows:

===== FEATURE BINNING (BEFORE) =====

Initial dataset shape: (440, 267)

Initial columns count: 267

===== BINNING SUMMARY =====

Number of new binned features: 3

- Age_Bin
- LivArea_Bin
- LotArea_Bin

===== AFTER FEATURE BINNING + ENCODING =====

Final dataset shape: (440, 278)

New columns added: 11

Sample of newly created columns:

```
['Age_Bin_VeryOld', 'LivArea_Bin_VeryHigh', 'LotArea_Bin_VeryLarge', 'LivArea_Bin_High', 'LotArea_Bin_Medium',  
'Age_Bin_Old', 'LivArea_Bin_Medium', 'Age_Bin_Recent', 'Age_Bin_MidAge', 'LotArea_Bin_Large']
```

Feature binning completed successfully.

Task 3: Exploratory Data Analysis (EDA)

3.1 Analysis and Interpretation: Analyse the dataset to answer specific questions:

- I. What is the distribution of sale prices?
- II. How do different features relate to sale price?
- III. Identify trends and patterns in the data.

```
# =====
```

```
# 3.1 Distribution of Sale Price
```

```
# =====
```

```
def plot_saleprice_distribution(self):
```

```
    if 'SalePrice' not in self.df.columns:
```

```
        print("SalePrice column not found.")
```

```
    return
```

```
plt.figure(figsize=(8, 5))
```

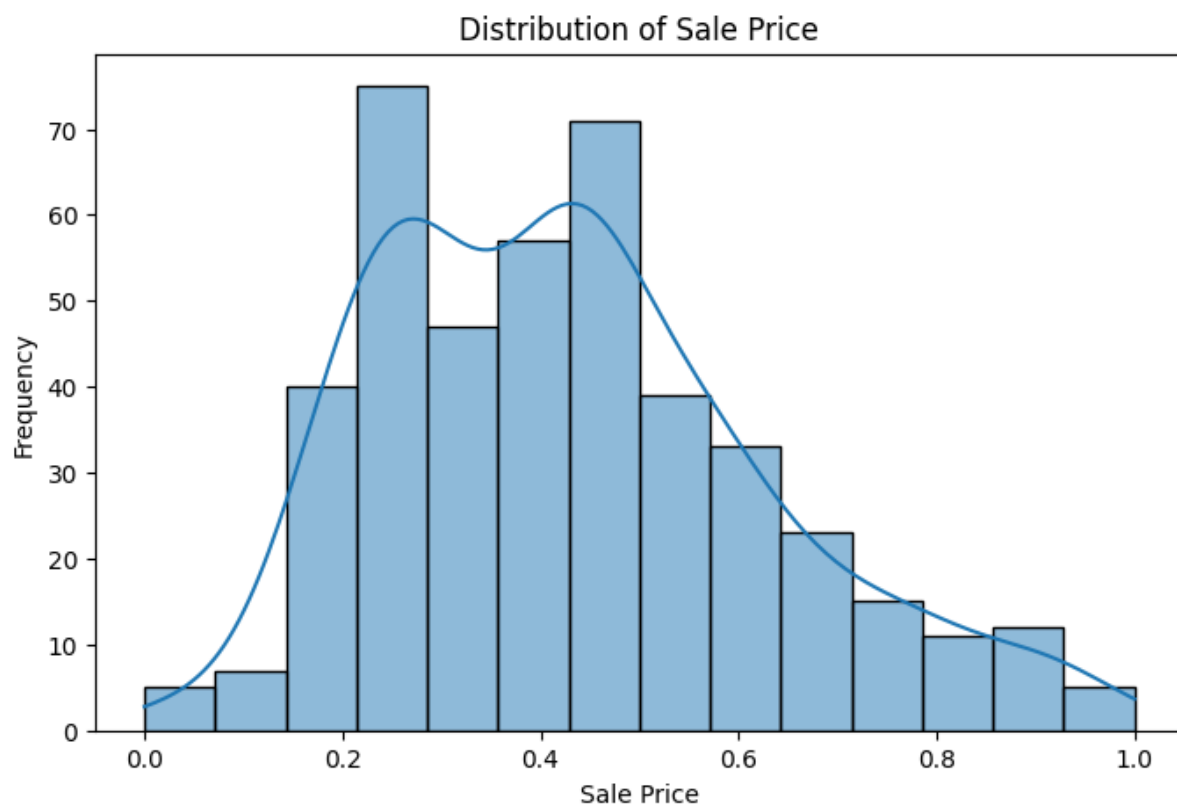
```
sns.histplot(self.df['SalePrice'], kde=True)
```

```
plt.title("Distribution of Sale Price")
```

```
plt.xlabel("Sale Price")
```

```
plt.ylabel("Frequency")
```

```
plt.show()
```



```
# =====

# 3.2 Relationship with Key Features

# =====

def plot_feature_relationships(self):

    target = 'SalePrice'

    if target not in self.df.columns:

        print("SalePrice column not found.")

        return

    # Select common important numeric features if available

    features = [

        'GrLivArea',

        'TotalBsmtSF',

        'OverallQual',

        'GarageArea',

        'YearBuilt',

        'LotArea'

    ]

    features = [f for f in features if f in self.df.columns]

    for col in features:

        plt.figure(figsize=(6, 4))

        sns.scatterplot(x=self.df[col], y=self.df[target])

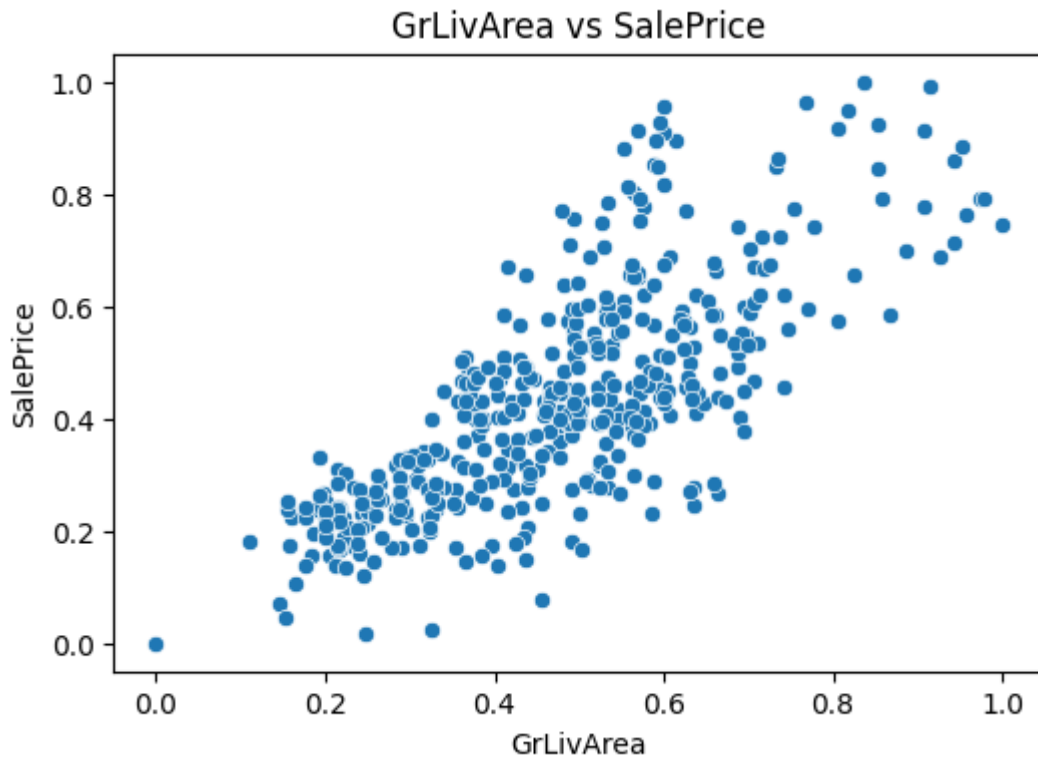
        plt.title(f"{col} vs SalePrice")

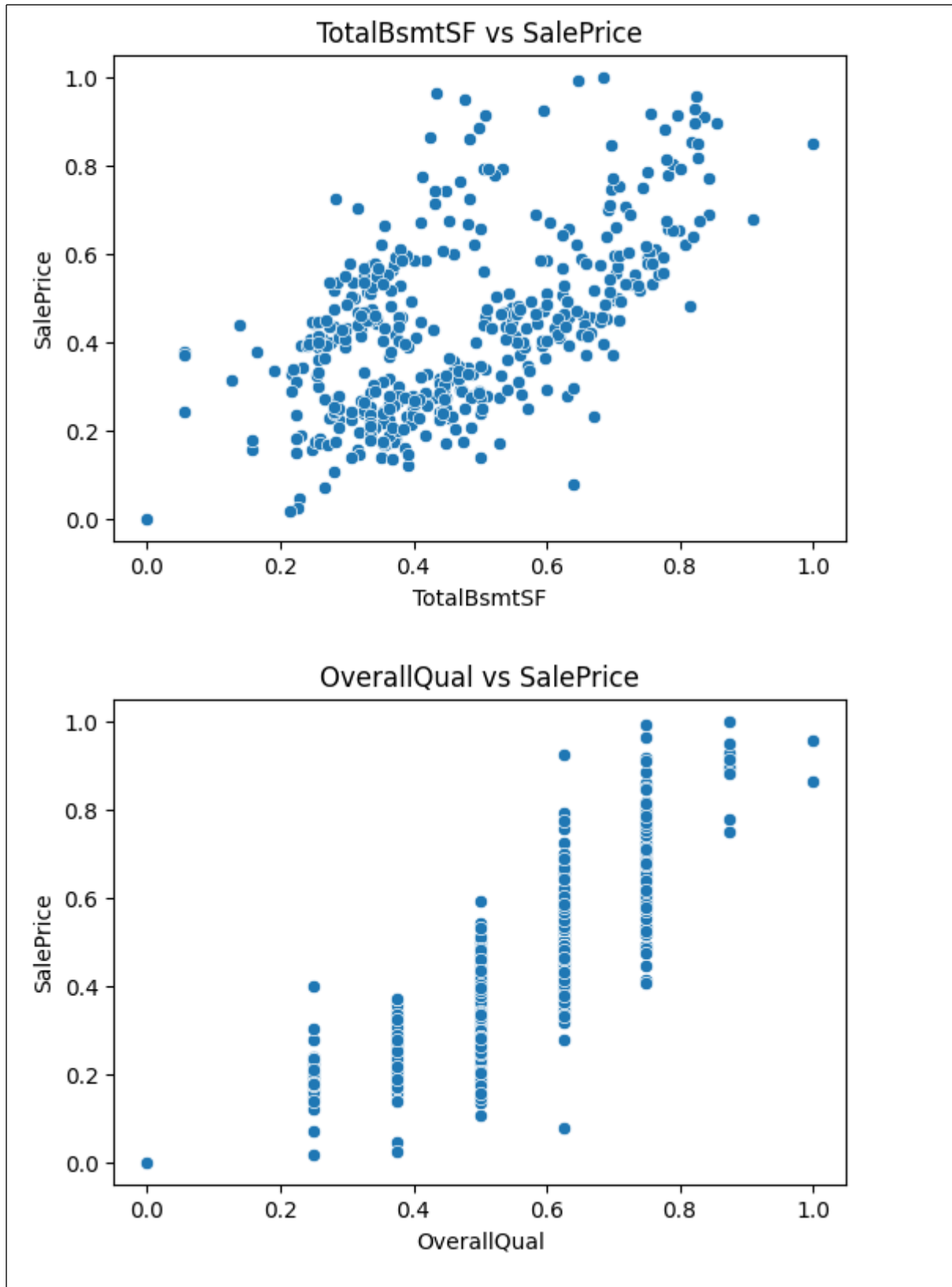
        plt.xlabel(col)

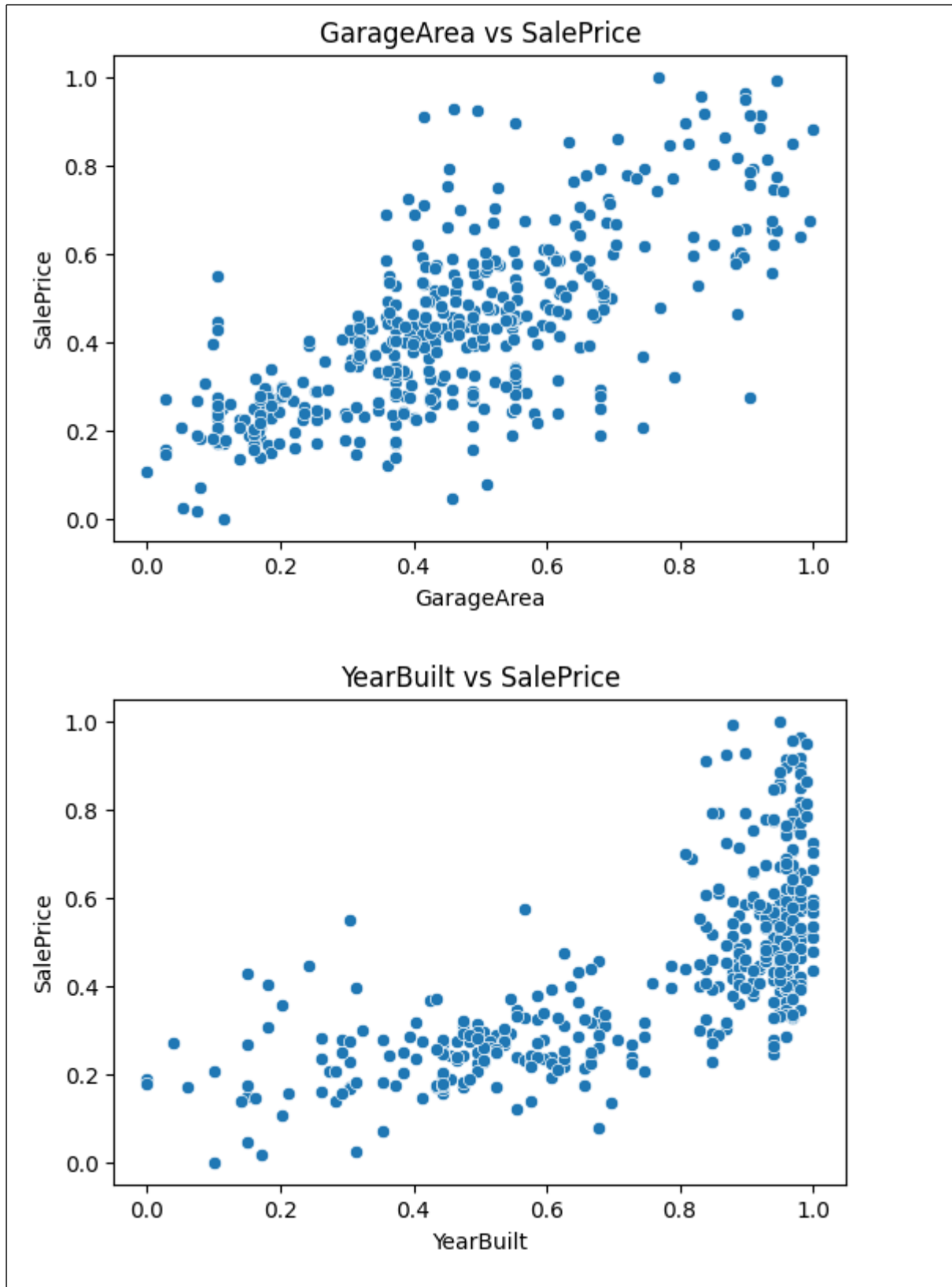
        plt.ylabel("SalePrice")

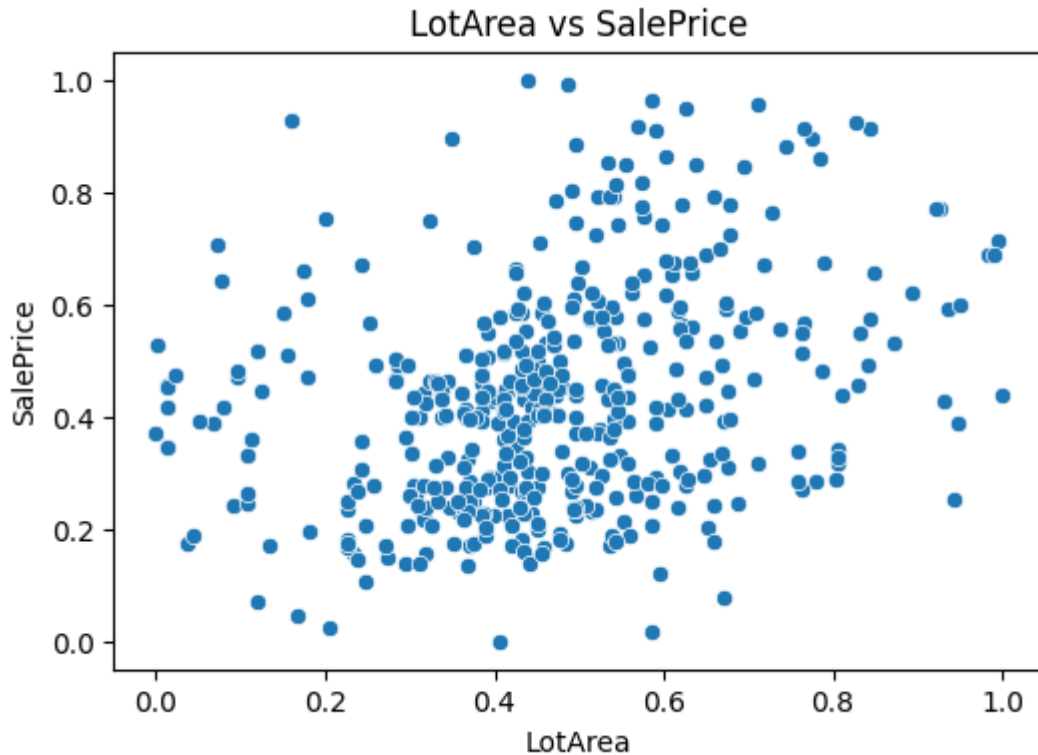
        plt.show()
```

The output is as follows:









```
# =====
```

```
# 3.3 Correlation Heatmap (Trends & Patterns)
```

```
# =====
```

```
def plot_correlation_heatmap(self):
```

```
    if 'SalePrice' not in self.df.columns:
```

```
        print("SalePrice column not found.")
```

```
    return
```

```
    # Select numeric columns only
```

```
    numeric_df = self.df.select_dtypes(include=['number'])
```

```
    corr_matrix = numeric_df.corr()
```

```
    plt.figure(figsize=(12, 8))
```

```
    sns.heatmap(
```

```
corr_matrix,
```

```
cmap='coolwarm',
```

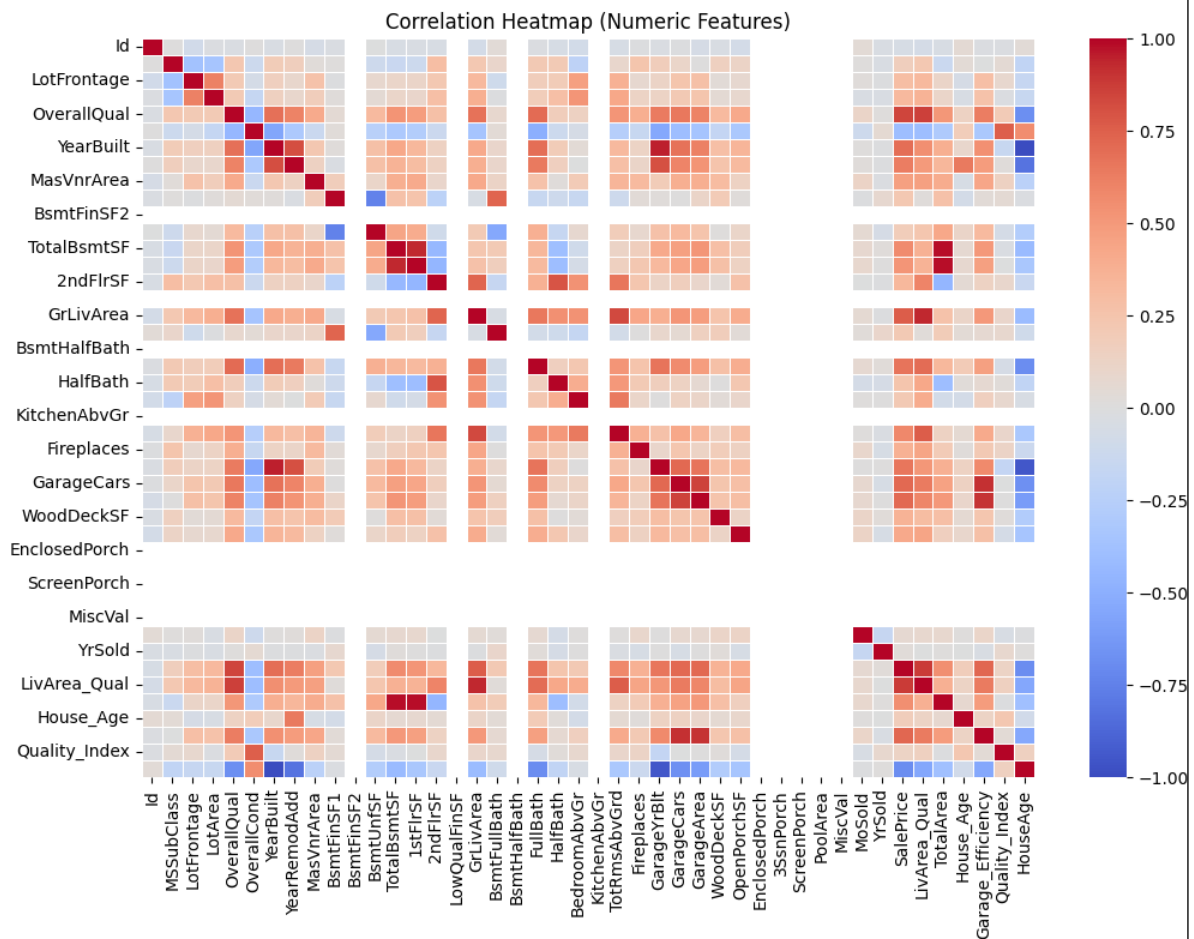
```
center=0,
```

```
linewidths=0.5
```

```
)
```

```
plt.title("Correlation Heatmap (Numeric Features)")
```

```
plt.show()
```



3.2 Visualisations: Create plots and charts to represent your findings clearly.

```
# =====
```

```
# 1. Sale Price Distribution
```

```
# =====
```

```
def plot_saleprice_distribution(self):
```

```
    if 'SalePrice' not in self.df.columns:
```

```
        print("SalePrice column not found.")
```

```
    return
```

```
plt.figure(figsize=(8, 5))
```

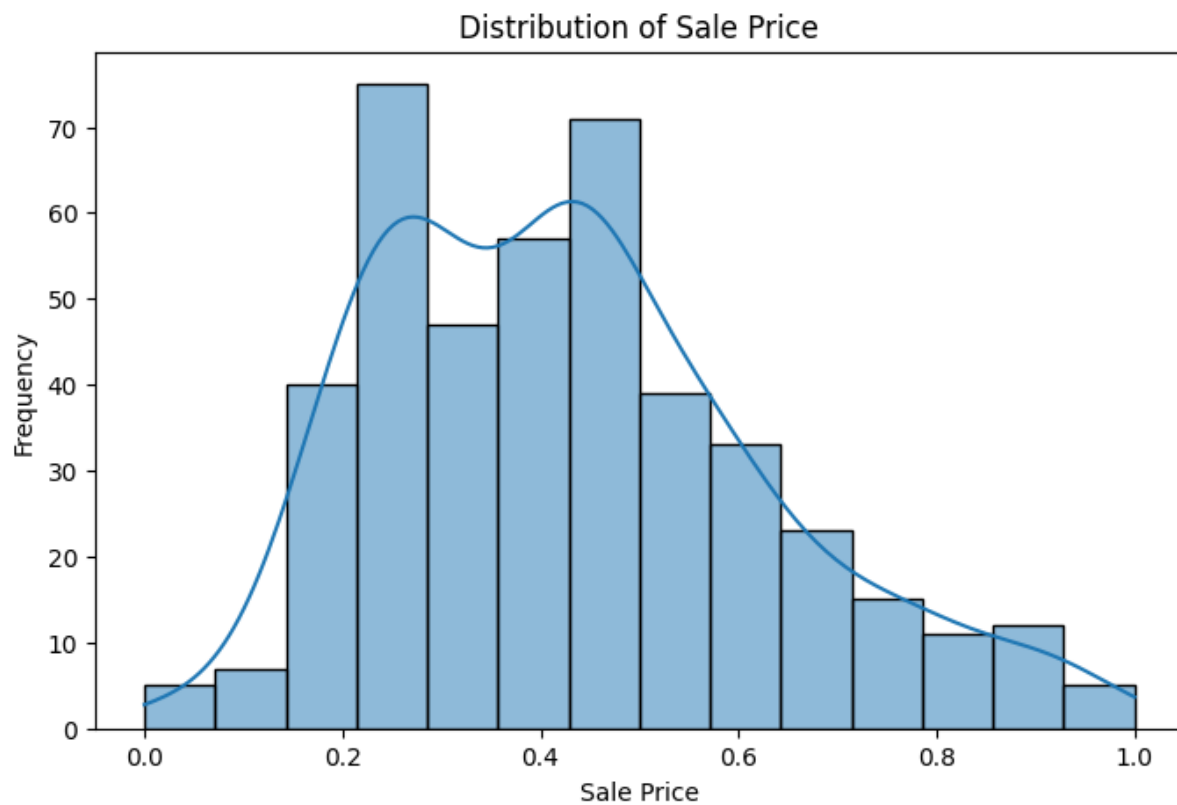
```
sns.histplot(self.df['SalePrice'], kde=True)
```

```
plt.title("Distribution of Sale Price")
```

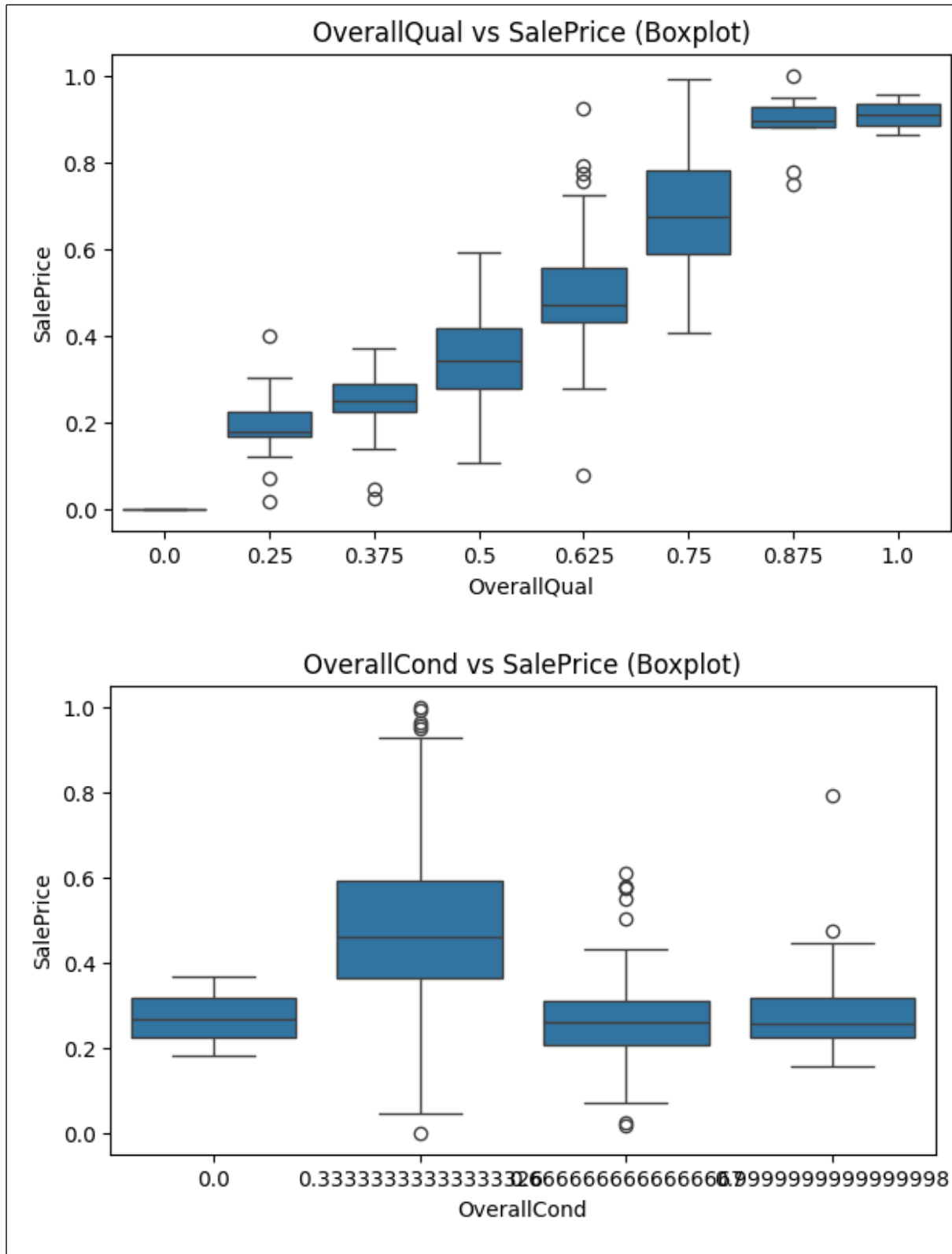
```
plt.xlabel("SalePrice")
```

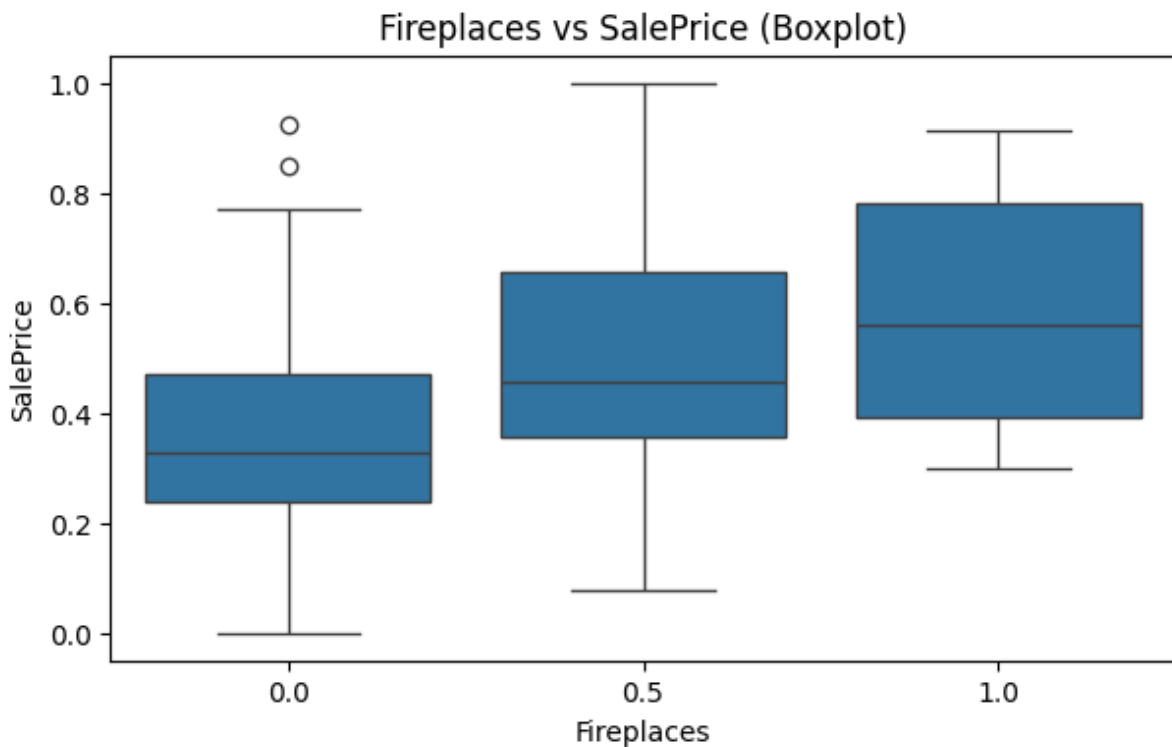
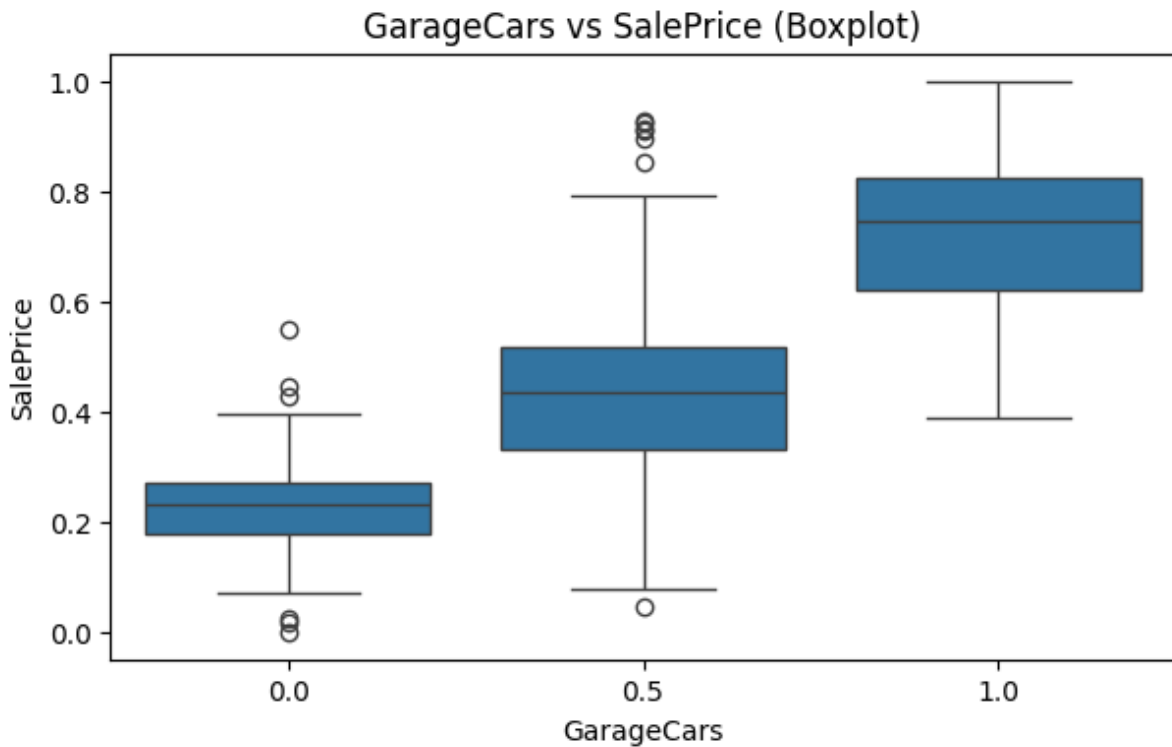
```
plt.ylabel("Frequency")
```

```
plt.show()
```



```
# =====  
  
# 3. Boxplots for Categorical Trends  
  
# =====  
  
def plot_categorical_trends(self):  
    target = 'SalePrice'  
  
    if target not in self.df.columns:  
        print("SalePrice column not found.")  
        return  
  
    cat_features = [  
        'OverallQual',  
        'OverallCond',  
        'GarageCars',  
        'Fireplaces'  
    ]  
  
    cat_features = [c for c in cat_features if c in self.df.columns]  
  
    for col in cat_features:  
        plt.figure(figsize=(7, 4))  
        sns.boxplot(x=self.df[col], y=self.df[target])  
        plt.title(f"{col} vs SalePrice (Boxplot)")  
        plt.xlabel(col)  
        plt.ylabel("SalePrice")  
        plt.show()
```

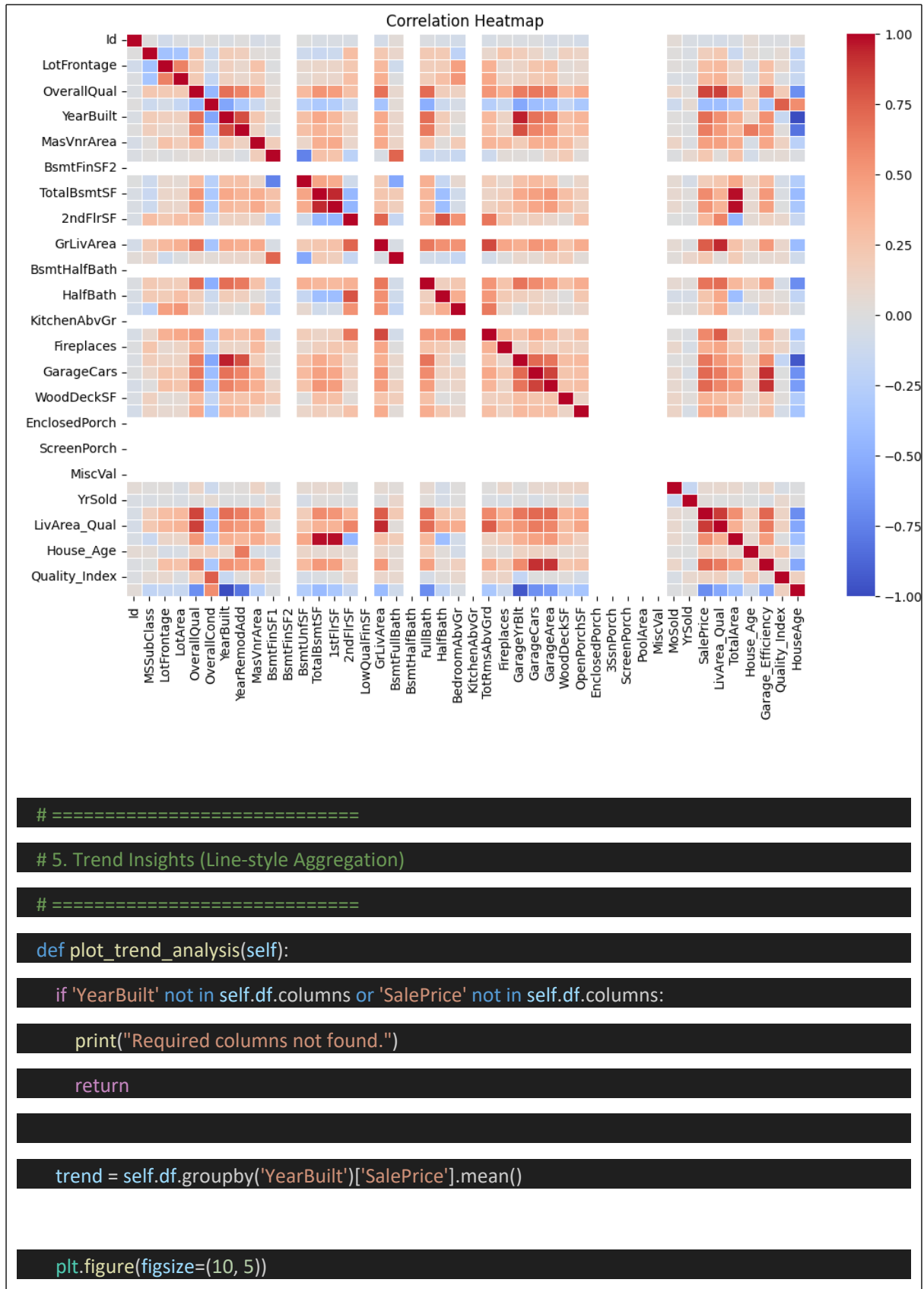




```
# =====
```

```
# 4. Correlation Heatmap
```

```
# =====  
  
def plot_correlation_heatmap(self):  
    numeric_df = self.df.select_dtypes(include=['number'])  
  
    plt.figure(figsize=(12, 8))  
    sns.heatmap(  
        numeric_df.corr(),  
        cmap='coolwarm',  
        center=0,  
        linewidths=0.5  
    )  
  
    plt.title("Correlation Heatmap")  
  
    plt.show()
```

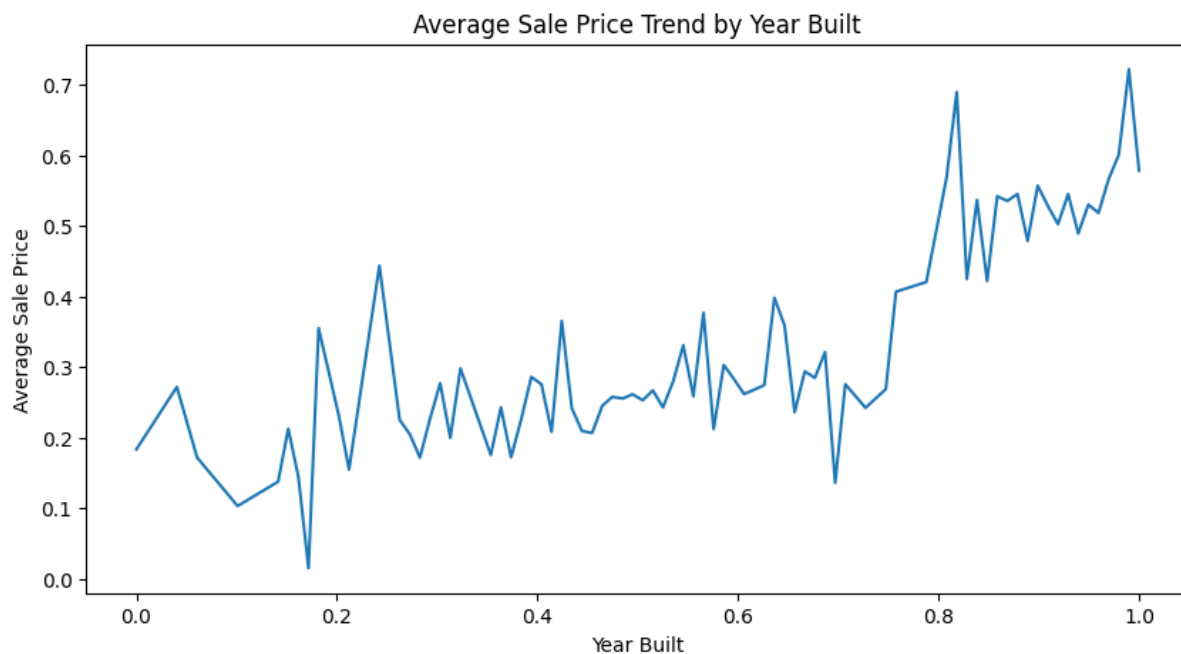
```
plt.plot(trend.index, trend.values)
```

```
plt.title("Average Sale Price Trend by Year Built")
```

```
plt.xlabel("Year Built")
```

```
plt.ylabel("Average Sale Price")
```

```
plt.show()
```



Task 4: Model Building and Evaluation

- I. Build models using XGBoost, LightGBM, and CatBoost to predict sale prices.
- II. Evaluate model performance using metrics such as RMSE, MAE, and R^2 .

Model Building

```
class HousePriceModelBuilder:
```

```
    def __init__(self, df, target_column):
```

```
        self.df = df.copy()
```

```
        self.target_column = target_column
```

```
self.X = None
```

```
self.y = None
```

```
self.X_train = None
```

```
self.X_test = None
```

```
self.y_train = None
```

```
self.y_test = None
```

```
self.models = {}
```

```
# =====
```

```
# Prepare Train-Test Split
```

```
# =====
```

```
def prepare_data(self, test_size=0.2, random_state=42):
```

```
self.X = self.df.drop(columns=[self.target_column])
```

```
self.y = self.df[self.target_column]
```

```
self.X_train, self.X_test, self.y_train, self.y_test = train_test_split(
```

```
self.X,
```

```
self.y,
```

```
test_size=test_size,
```

```
random_state=random_state
```

```
)
```

```
print("\nData split completed successfully.")
```

```
print("X_train shape:", self.X_train.shape)
```

```
print("X_test shape:", self.X_test.shape)
```

```
return self.X_train, self.X_test, self.y_train, self.y_test
```

```
# =====
```

```
# Train XGBoost
```

```
# =====
```

```
def build_xgboost(self):
```

```
    model = XGBRegressor(
```

```
        n_estimators=300,
```

```
        learning_rate=0.05,
```

```
        max_depth=6,
```

```
        subsample=0.8,
```

```
        colsample_bytree=0.8,
```

```
        random_state=42
```

```
    )
```

```
    model.fit(self.X_train, self.y_train)
```

```
    self.models['XGBoost'] = model
```

```
    print("\nXGBoost model built successfully.")
```

```
    return model
```

```
# =====
```

```
# Train LightGBM
```

```
# =====
```

```
def build_lightgbm(self):
```

```
model = LGBMRegressor(  
    n_estimators=300,  
    learning_rate=0.05,  
    random_state=42  
)
```

```
model.fit(self.X_train, self.y_train)
```

```
self.models['LightGBM'] = model
```

```
print("\nLightGBM model built successfully.")
```

```
return model
```

```
# =====
```

```
# Train CatBoost
```

```
# =====
```

```
def build_catboost(self):
```

```
model = CatBoostRegressor(  
    iterations=300,  
    learning_rate=0.05,  
    depth=6,  
    loss_function='RMSE',  
    verbose=0  
)
```

```
model.fit(self.X_train, self.y_train)
```

```
self.models['CatBoost'] = model
```

```
print("\nCatBoost model built successfully.")
```

```
return model
```

```
# =====
```

```
# Get all models
```

```
# =====
```

```
def get_models(self):
```

```
print("\n==== MODELS BUILT ====")
```

```
if not self.models:
```

```
    print("No models have been trained yet.")
```

```
else:
```

```
    for name in self.models.keys():
```

```
        print(f"- {name}")
```

```
return self.models
```

```
# =====
```

```
# Get train/test splits
```

```
# =====
```

```
def get_data_splits(self):
```

```
return self.X_train, self.X_test, self.y_train, self.y_test
```

The output is as follows:

Data split completed successfully.

X_train shape: (352, 277)

X_test shape: (88, 277)

XGBoost model built successfully.

[LightGBM] [Warning] Found whitespace in feature_names, replace with underlines

[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.001155 seconds.

You can set `force\_row\_wise=true` to remove the overhead.

And if memory is not enough, you can set `force\_col\_wise=true`.

[LightGBM] [Info] Total Bins 1880

[LightGBM] [Info] Number of data points in the train set: 352, number of used features: 102

[LightGBM] [Info] Start training from score 0.431369

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

- LightGBM

- CatBoost

===== MODELS BUILT =====

- XGBoost

- LightGBM

- CatBoost

Evaluation

```
class ModelEvaluator:
    def __init__(self, models, X_test, y_test):
        self.models = models
        self.X_test = X_test
        self.y_test = y_test
        self.results = {}

    # =====
    # Evaluate all models
    # =====

    def evaluate(self):
        for name, model in self.models.items():
            y_pred = model.predict(self.X_test)

            mae = mean_absolute_error(self.y_test, y_pred)
            rmse = np.sqrt(mean_squared_error(self.y_test, y_pred))
            r2 = r2_score(self.y_test, y_pred)

            self.results[name] = {
                "MAE": mae,
                "RMSE": rmse,
```

```
"R2 Score": r2
```

```
}
```

```
return self.results
```

```
# =====
```

```
# Display results as DataFrame
```

```
# =====
```

```
def show_results(self):
```

```
    results_df = pd.DataFrame(self.results).T
```

```
    print("\nModel Performance Comparison:\n")
```

```
    print(results_df)
```

```
return results_df
```

```
# =====
```

```
# Get best model (based on R2 Score)
```

```
# =====
```

```
def get_best_model(self):
```

```
    best_model = max(self.results, key=lambda x: self.results[x]["R2 Score"])
```

```
    print("\nBest Model:", best_model)
```

```
    print("R2 Score:", self.results[best_model]["R2 Score"])
```

```
return best_model, self.models[best_model]
```

```
# =====
```

```
# Detailed evaluation of one model
```

```
# =====
```

```
def evaluate_single_model(self, model_name):

    if model_name not in self.models:

        print("Model not found!")

        return

    model = self.models[model_name]

    y_pred = model.predict(self.X_test)

    mae = mean_absolute_error(self.y_test, y_pred)

    rmse = np.sqrt(mean_squared_error(self.y_test, y_pred))

    r2 = r2_score(self.y_test, y_pred)

    print(f"\n{model_name} Performance:")

    print(f"MAE : {mae}")

    print(f"RMSE : {rmse}")

    print(f"R2 : {r2}")

    return mae, rmse, r2
```

Model Performance Comparison:

	MAE	RMSE	R2 Score
XGBoost	0.038917	0.052174	0.940348
LightGBM	0.042403	0.058290	0.925542
CatBoost	0.041033	0.055836	0.931681

Best Model: XGBoost

R2 Score: 0.9403480463622353

Best Model: XGBRegressor(base_score=None, booster=None, callbacks=None,

```

colsample_bylevel=None, colsample_bynode=None,
colsample_bytree=0.8, device=None, early_stopping_rounds=None,
enable_categorical=False, eval_metric=None, feature_types=None,
feature_weights=None, gamma=None, grow_policy=None,
importance_type=None, interaction_constraints=None,
learning_rate=0.05, max_bin=None, max_cat_threshold=None,
max_cat_to_onehot=None, max_delta_step=None, max_depth=6,
max_leaves=None, min_child_weight=None, missing=nan,
monotone_constraints=None, multi_strategy=None, n_estimators=300,
n_jobs=None, num_parallel_tree=None, ...)

```

Model saved successfully as house_price_best_model.pkl

Model loaded successfully from house_price_best_model.pkl

Best Model Loaded from Pickle:

```

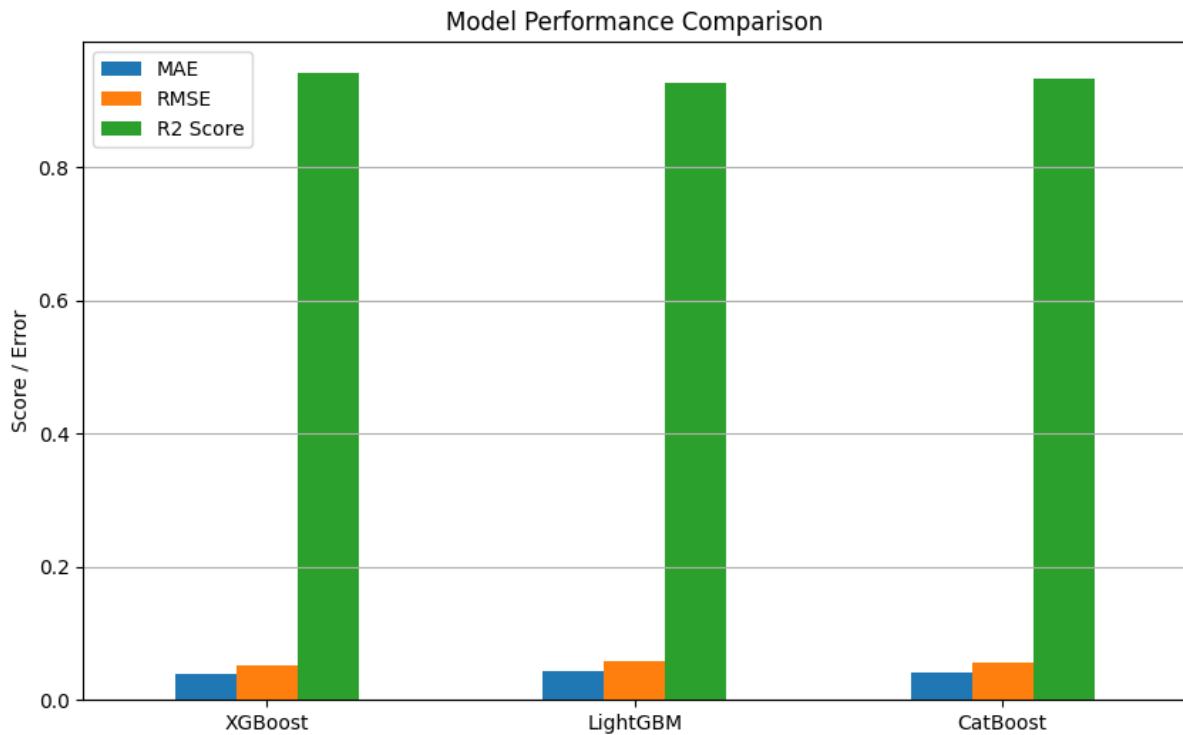
XGBRegressor(base_score=None, booster=None, callbacks=None,
colsample_bylevel=None, colsample_bynode=None,
colsample_bytree=0.8, device=None, early_stopping_rounds=None,
enable_categorical=False, eval_metric=None, feature_types=None,
feature_weights=None, gamma=None, grow_policy=None,
importance_type=None, interaction_constraints=None,
learning_rate=0.05, max_bin=None, max_cat_threshold=None,
max_cat_to_onehot=None, max_delta_step=None, max_depth=6,
max_leaves=None, min_child_weight=None, missing=nan,
monotone_constraints=None, multi_strategy=None, n_estimators=300,
n_jobs=None, num_parallel_tree=None, ...)

```

===== MODEL PERFORMANCE REPORT =====

	MAE	RMSE	R2 Score
XGBoost	0.038917	0.052174	0.940348
LightGBM	0.042403	0.058290	0.925542
CatBoost	0.041033	0.055836	0.931681

Best Model Based on R2 Score: XGBoost



Cleaned data is saved at: data\cleaned_house_prices.csv

===== BEST MODEL SUMMARY =====

Model: XGBoost

MAE 0.038917

RMSE 0.052174

R2 Score 0.940348

Name: XGBoost, dtype: float64

===== TOP FACTORS INFLUENCING SALE PRICE =====

LivArea_Qual 0.879580

OverallQual 0.844672

GrLivArea 0.760636
Garage_Efficiency 0.726454
GarageCars 0.717953
GarageArea 0.712075
YearBuilt 0.688723
FullBath 0.664611
GarageYrBltn 0.660210
YearRemodAdd 0.627344
Name: SalePrice, dtype: float64

===== DYNAMIC DECISION INSIGHTS =====

- Liv Area Qual has a very strong positive relationship with sale price (correlation: 0.88).

Decision: Homes with better liv area qual are likely to have higher market value.

- Overall Qual has a very strong positive relationship with sale price (correlation: 0.84).

Decision: Homes with better overall qual are likely to have higher market value.

- Gr Liv Area has a very strong positive relationship with sale price (correlation: 0.76).

Decision: Homes with better gr liv area are likely to have higher market value.

- Garage Efficiency has a strong positive relationship with sale price (correlation: 0.73).

Decision: Homes with better garage efficiency are likely to have higher market value.

- Garage Cars has a strong positive relationship with sale price (correlation: 0.72).

Decision: Homes with better garage cars are likely to have higher market value.

- Garage Area has a strong positive relationship with sale price (correlation: 0.71).

Decision: Homes with better garage area are likely to have higher market value.

- Year Built has a strong positive relationship with sale price (correlation: 0.69).

Decision: Homes with better year built are likely to have higher market value.

- Full Bath has a strong positive relationship with sale price (correlation: 0.66).

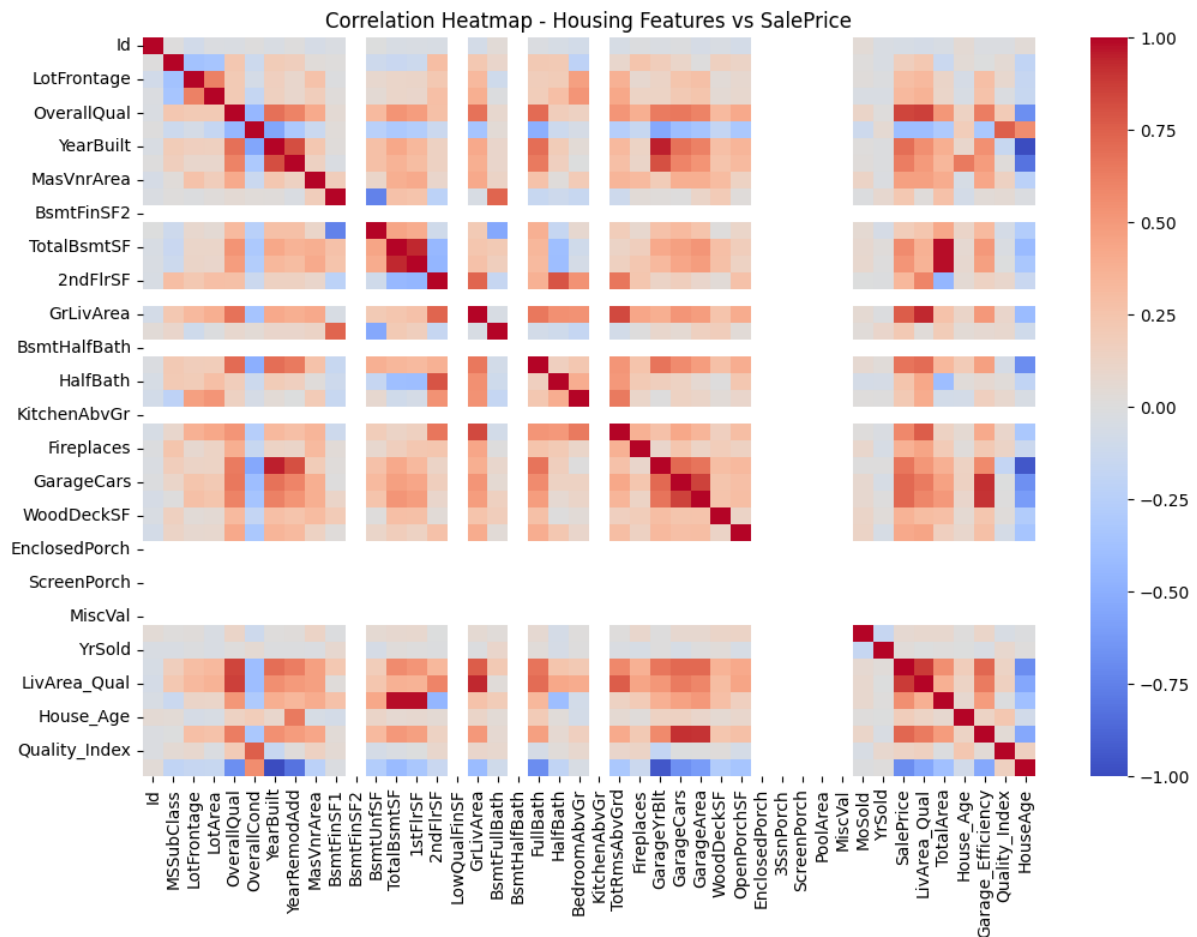
Decision: Homes with better full bath are likely to have higher market value.

- Garage Yr Blt has a strong positive relationship with sale price (correlation: 0.66).

Decision: Homes with better garage yr blt are likely to have higher market value.

- Year Remod Add has a strong positive relationship with sale price (correlation: 0.63).

Decision: Homes with better year remod add are likely to have higher market value.



Final note

===== FINAL HOUSING DECISION RECOMMENDATION =====

Users should prioritize properties with:

- Strong liv area qual (impact score: 0.88)
- Strong overall qual (impact score: 0.84)
- Strong gr liv area (impact score: 0.76)
- Strong garage efficiency (impact score: 0.73)
- Strong garage cars (impact score: 0.72)

These factors show the strongest relationship with higher housing prices and resale potential.

THE END